

Materiais e métodos — BCI de imagética motora para acionamento de órtese

Texto original com os dados corrigidos contra o código

1 Materiais e métodos

1.1 Aquisição dos sinais EEG

A aquisição ao vivo é feita com um amplificador OpenBCI Cyton de oito canais e conversor de 24 bits, conectado ao computador por um *dongle* serial e acessado pela biblioteca BrainFlow. No *dongle* USB a taxa de amostragem é fixa em 250 Hz; as taxas superiores previstas na placa exigem o módulo Wi-Fi, que não é utilizado.

Três eletrodos são empregados, posicionados segundo o sistema 10–20 sobre o córtex sensório-motor: C3 no canal físico 1, C4 no canal 2 e Cz no canal 3. C3 e C4 formam o par simétrico onde a lateralização do ERD se manifesta [4], e Cz serve de referência central da linha média. A restrição a três eletrodos é deliberada, por viabilizar uma montagem de baixo custo, e é também o principal limitante do desempenho, como discutido na Seção 1.10.

O protocolo de captura implementado em `capturar.py` apresenta ao usuário, em tela cheia, uma dica visual por segmento e grava a sessão inteira em fluxo contínuo, a 250 Hz. Cada execução contém 15 repetições de cada mão, sorteadas em ordem aleatória, com um segmento de descanso de 4,1 s antes de cada tarefa e 2 s de estabilização do fluxo logo após o início da transmissão. São 60 segmentos de 4,1 s, cerca de 4,1 minutos de gravação. Entre os segmentos não há pausa: eles são contíguos, porque as janelas de análise precisam de metade do segmento anterior e qualquer intervalo não gravado cairia dentro delas. Foram realizadas duas sessões, ambas de imagética e ambas com as três classes: mão direita, mão esquerda e repouso. Antes de gravar, cada canal passa por remoção de tendência linear por partes, com as quebras nos pontos médios entre eventos, e assim cada janela de análise recebe a sua própria reta e o degrau entre retas cai na borda da janela, nunca no centro; as demais

etapas de filtragem ficam desligadas nesse ponto, porque a filtragem definitiva é aplicada de forma idêntica a todas as fontes de dados na cadeia descrita a seguir. Cada sessão é gravada em dois arquivos CSV, `continuo.csv` e `eventos.csv`, e o código da classe fica na coluna `code` do segundo, de onde o rótulo é recuperado posteriormente.

1.2 Conjuntos de dados

O treinamento combina duas origens. A principal é o *PhysioNet EEG Motor Movement/Imagery Dataset* [2], adquirido com o sistema BCI2000 [7], do qual se utilizam 105 sujeitos e os seis *runs* de mão: 3, 7 e 11, de movimento real, e 4, 8 e 12, de imagética. Os eventos anotados são T0 (repouso), T1 (mão esquerda) e T2 (mão direita), o que dá as três classes do problema. Essa seleção percorre 630 arquivos e produz 18.266 janelas: a primeira anotação de cada *run* é um T0 no instante zero, sem segmento anterior para formar a metade esquerda da janela, e por isso é descartada. A segunda origem são as gravações do próprio usuário, das quais se aproveitam 87 janelas centradas; outras três caem pelo mesmo motivo. O pacote de distribuição traz um subconjunto de dez sujeitos e duas sessões gravadas, suficiente para reproduzir a cadeia e rodar um treino de demonstração, mas não para reconstruir o modelo completo.

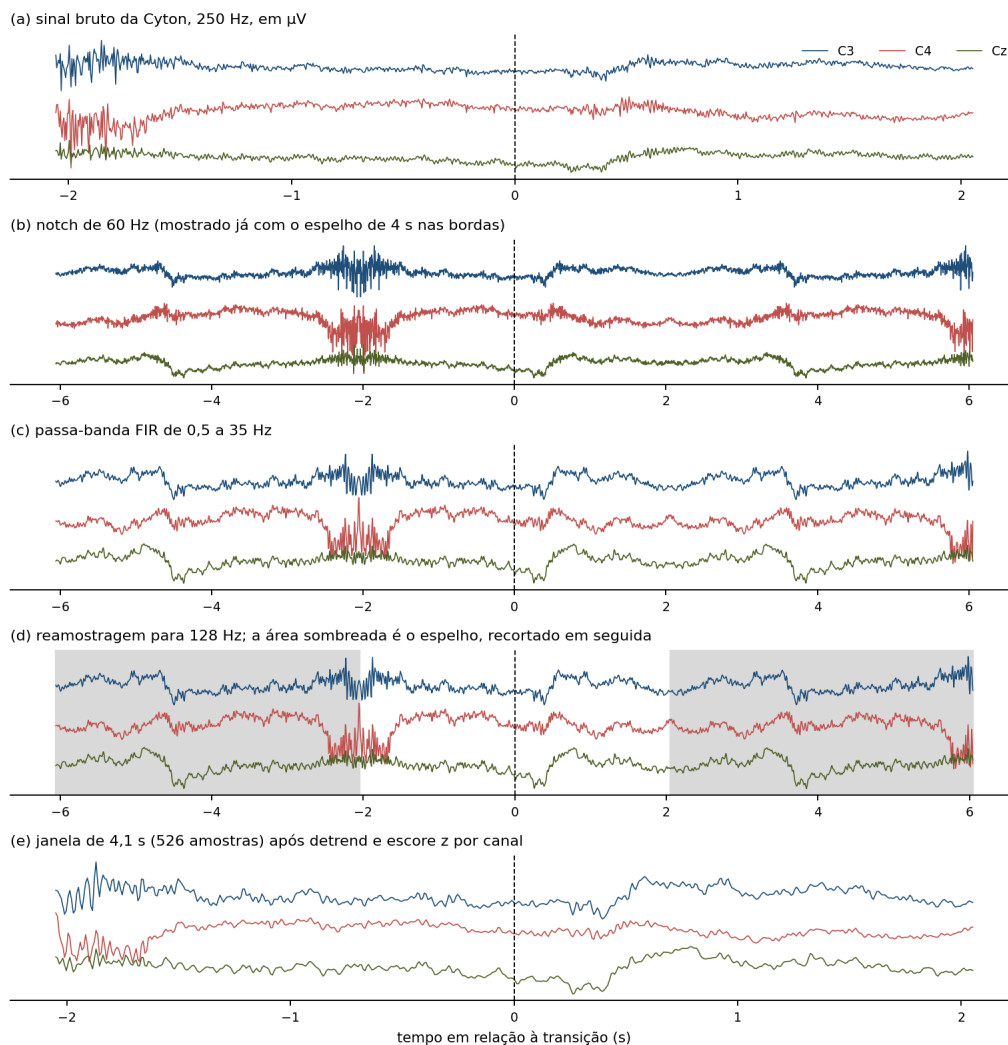
Quatro sujeitos do conjunto público são descartados sempre. A auditoria dos arquivos mostrou que o padrão é 160 Hz, cerca de 123 a 125 s e 30 eventos por *run*, e que os sujeitos S088 e S092 foram gravados a 128 Hz com 38 eventos, o S100 a 128 Hz com 24 eventos, e o *run* R03 do S089 tem 181 s e 44 eventos. A taxa divergente é o problema mais grave, porque desalinha a duração das épocas. A exclusão reduz os 109 sujeitos do `eegmmidb` aos 105 utilizados e é aplicada em um único ponto do código, para não se perder caso a faixa de sujeitos seja ampliada.

O conjunto final tem 18.353 amostras, distribuídas em 8.860 de repouso, 4.773 de mão esquerda e 4.720 de mão direita. O desbalanceamento é intrínseco ao protocolo do conjunto público: cada tentativa de tarefa é seguida de um intervalo de repouso, de modo que a classe majoritária tem aproximadamente o dobro de exemplos de cada classe motora.

1.3 Cadeia de pré-processamento

A cadeia de pré-processamento obedece a uma restrição de projeto: ela é idêntica para os arquivos EDF do conjunto público e para os CSV capturados ao vivo. Se as duas fontes fossem tratadas de forma diferente, a rede seria treinada em um domínio e operaria em outro. Por isso a cadeia está implementada no módulo `nucleo/preproc.py`, cuja função `cadeia_mne()` é chamada tanto pela leitura de EDF quanto pela função `janela_bruta()`, que atende a leitura de janelas capturadas e a inferência em tempo real. A Figura 1 mostra o efeito de cada etapa sobre uma gravação real.

Figura 1: Cadeia de pré-processamento aplicada a uma gravação real do usuário. O trecho destacado em (d) é o preenchimento por reflexão discutido na Seção 1.3.2.



Fonte: o autor (2026).

As etapas são: filtro *notch* de 60 Hz, para a interferência da rede elétrica; passa-

banda FIR de 0,5 a 35 Hz, projetado por janelamento (*firwin*); reamostragem para 128 Hz, taxa em que a rede opera e para a qual convergem os 250 Hz da Cyton e os 160 Hz do conjunto público; recorte da janela de $-2,05$ s a $+2,05$ s em torno da transição entre segmentos, sem correção de linha de base em nenhuma das classes; remoção de tendência linear; e normalização pelo escore z por canal, conforme a Equação 1, em que μ_c e σ_c são a média e o desvio padrão do canal c .

$$z_c[n] = \frac{x_c[n] - \mu_c}{\sigma_c} \quad (1)$$

A normalização é o ponto em que treino e operação podem divergir sem que nada acuse a diferença. A média e o desvio padrão são calculados apenas sobre o conjunto de treino e gravados dentro do arquivo do modelo. Na inferência, a função `normalizar_com_modelo()` recupera essas mesmas estatísticas do arquivo e as aplica ao sinal novo, em vez de recalculá-las sobre o lote corrente. Assim, um sinal com amplitude atípica não é reescalado para parecer normal, e a rede vê a mesma escala em que foi treinada.

1.3.1 Remoção da referência média comum

A versão anterior do sistema aplicava referência média comum (CAR) após selecionar C3, C4 e Cz. Essa etapa foi removida das duas pontas da cadeia, e a razão é mensurável. Com apenas três eletrodos, a CAR impõe $x_{C3} + x_{C4} + x_{Cz} = 0$ em cada instante, e o posto da matriz de sinais cai de três para dois. A decomposição em valores singulares de uma janela real mostrou, com a CAR aplicada, os dois primeiros valores singulares na ordem de 4×10^{-4} e o terceiro na ordem de 10^{-12} ; sem a CAR, o terceiro valor volta à mesma ordem de grandeza dos demais. Oito ordens de grandeza abaixo dos outros dois, o terceiro componente é numericamente nulo, e a rede recebia três canais com apenas duas dimensões de informação.

A CAR só aproxima uma referência neutra quando há cobertura de toda a cabeça, o que uma montagem de três eletrodos centrais não oferece. Aplicá-la apenas ao conjunto público, que tem 64 canais, resolveria o problema numérico mas violaria a restrição de cadeia única, então a etapa foi retirada dos dois lados.

1.3.2 Tratamento das janelas curtas

As janelas de análise têm 4,1 s a 128 Hz e as duas origens chegam a esse comprimento por caminhos diferentes. Duas correções são necessárias. A primeira é de borda: antes da filtragem aplica-se preenchimento por reflexão de 4 s em cada extremidade, porque o filtro FIR de corte inferior em 0,5 Hz precisa de contexto para não distorcer os extremos da janela. Esse preenchimento é recortado logo após a filtragem, e o recorte é exato por construção: qualquer que seja a taxa de origem, os 4 s equivalem a $4 \times 128 = 512$ amostras de cada lado depois da reamostragem, um número inteiro. Nenhuma amostra espelhada nessa etapa chega à rede. A segunda correção é de comprimento: $4,1 \times 128 = 524,8$ não é inteiro, e o projeto fixa 526 amostras contando as duas pontas, enquanto o `mne.Epochs` devolve 525 para o EDF. A função `ajustar_tamanho()` completa o início por reflexão, mantendo intacto o fim da janela, que contém a tarefa.

Medido sobre os arquivos do pacote, esse ajuste completa exatamente uma amostra por janela do EDF, ou 0,19 % do comprimento, e nenhuma amostra nas gravações da Cyton e no laço de tempo real, que chegam aos 526 pontos por conta própria. A mesma função recusa a janela com erro se faltar mais de 2 % do comprimento, para que uma janela montada errado não passe despercebida. O laço ao vivo solicita à placa a quantidade de amostras que corresponde à janela completa do modelo, como descrito na Seção 1.9, e por isso não há o que completar. Uma versão anterior deste projeto gravava um arquivo por tentativa e deixava um vão de cerca de 1,4 s entre elas, e boa parte de cada janela de movimento tinha então de ser fabricada por espelhamento; a gravação contínua eliminou esse problema.

1.4 Construção das amostras

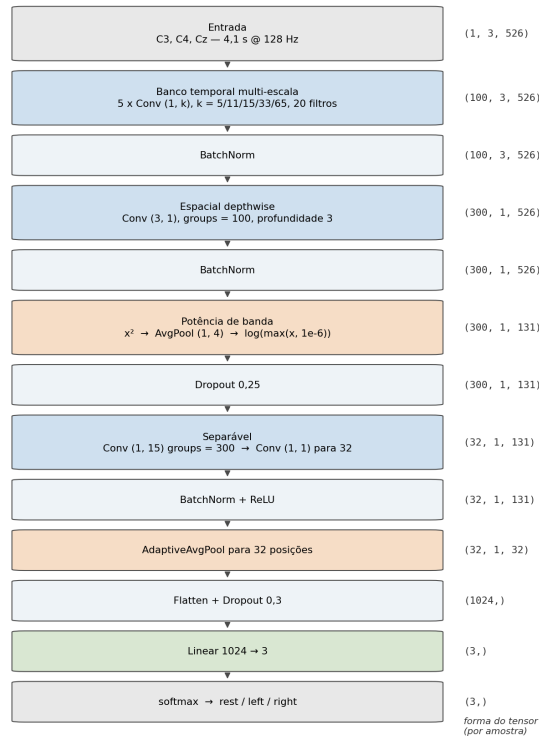
Cada amostra é uma matriz de três linhas, na ordem C3, C4 e Cz, por 526 colunas, correspondentes a 4,1 s amostrados a 128 Hz. O rótulo vem da anotação do evento, no caso dos arquivos públicos, ou do código registrado em `eventos.csv`, no caso das gravações próprias, segundo o mapeamento RE → repouso, LH → esquerda e RH → direita. O tensor entregue à rede tem forma $(N, 3, 526)$ e recebe uma dimensão de canal unitário internamente, passando a $(N, 1, 3, 526)$, de modo que as convoluções

bidimensionais tratem o eixo dos eletrodos e o eixo do tempo separadamente.

1.5 Arquitetura da rede

O classificador opera diretamente sobre a forma de onda, sem extração manual de características. A Figura 2 mostra a sequência de camadas com as formas dos tensores lidas do próprio modelo, e o Quadro 1 detalha cada operação com seus parâmetros.

Figura 2: Camadas da rede e formas dos tensores em cada estágio.



Fonte: o autor (2026).

1.5.1 Banco temporal multi-escala

A primeira camada é um banco de filtros aprendidos, organizado em cinco escalas paralelas de 20 filtros cada. Cada filtro percorre o eixo do tempo de um eletrodo por vez, sem misturar eletrodos. Para um kernel w de comprimento k aplicado ao canal c , a saída no instante n é dada pela Equação 2:

$$h_f[c, n] = \sum_{j=0}^{k-1} w_f[j] x[c, n + j - \lfloor k/2 \rfloor], \quad (2)$$

Quadro 1: Arquitetura da CNN, camada a camada. Formas de saída por amostra, sem a dimensão de lote.

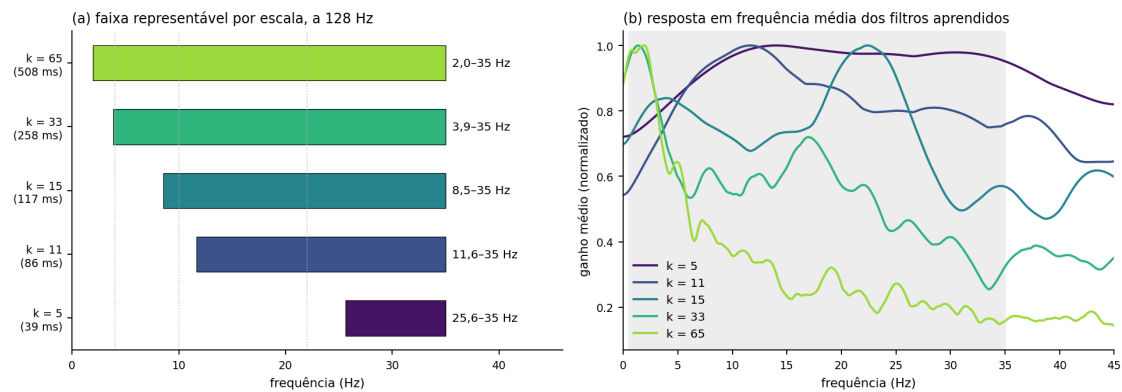
#	Camada	Operação	Param.	Saída
0	Entrada	C3, C4 e Cz, 4,1 s a 128 Hz	0	(1, 3, 526)
1	Banco temporal multi-escala	5 convoluções paralelas $(1,k)$, $k \in \{5, 11, 15, 33, 65\}$, 20 filtros cada, preenchimento $k/2$, sem viés	2.580	(100, 3, 526)
2	Normalização em lote	<i>BatchNorm</i> sobre 100 mapas	200	(100, 3, 526)
3	Espacial <i>depthwise</i>	convolução (3,1) com <i>groups</i> = 100 e profundidade 3, sem viés	900	(300, 1, 526)
4	Normalização em lote	<i>BatchNorm</i> sobre 300 mapas	600	(300, 1, 526)
5	Quadrado	x^2 , elemento a elemento	0	(300, 1, 526)
6	Média móvel	<i>average pooling</i> (1,4)	0	(300, 1, 131)
7	Logaritmo com piso	$\log(\max(x, 10^{-6}))$	0	(300, 1, 131)
8	<i>Dropout</i> 0,25	regularização	0	(300, 1, 131)
9	Separável (temporal)	convolução (1,15) com <i>groups</i> = 300, preenchimento 7, sem viés	4.500	(300, 1, 131)
10	Separável (pontual)	convolução (1,1) de 300 para 32 mapas, sem viés	9.600	(32, 1, 131)
11	Normalização em lote e ReLU	<i>BatchNorm</i> e retificação	64	(32, 1, 131)
12	<i>Pooling</i> adaptativo	média para 32 posições temporais	0	(32, 1, 32)
13	Achatamento e <i>dropout</i> 0,3	vetor de características	0	(1024,)
14	Camada linear	projeção para as três classes	3.075	(3,)
Total de parâmetros treináveis			21.519	

Fonte: o autor (2026), a partir de `nucleo/modelo.py` e `config.py`.

em que o deslocamento $\lfloor k/2 \rfloor$ decorre do preenchimento simétrico, que preserva o comprimento da janela. Os cinco ramos são concatenados no eixo de mapas, produzindo 100 mapas temporais.

O motivo de usar escalas diferentes está na relação entre o comprimento do kernel e a frequência que ele consegue representar, e essa relação é frequentemente lida ao contrário. O comprimento limita a frequência mais *baixa* representável, não a mais alta: um ciclo inteiro precisa caber dentro do kernel. A 128 Hz, um kernel de comprimento k representa a partir de $128/k$ Hz, de modo que são os ramos curtos que alcançam o teto de 35 Hz da banda. O painel (a) da Figura 3 mostra a faixa coberta por cada escala e o painel (b), a resposta em frequência média dos filtros aprendidos, extraída dos pesos do modelo treinado.

Figura 3: Banco de filtros multi-escala: faixa representável por escala e resposta em frequência dos filtros aprendidos.



Fonte: o autor (2026).

O Quadro 2 relaciona cada escala ao seu limite inferior. O ramo mais longo, de 65 amostras, alcança 2,0 Hz e cobre a faixa lenta em que se manifestam os potenciais corticais relacionados a movimento; o mais curto, de 5 amostras, cobre a porção alta de β .

1.5.2 Filtro espacial *depthwise*

A segunda camada colapsa os três eletrodos em uma única linha, mas de forma agrupada: cada um dos 100 mapas temporais recebe seus próprios três filtros espaciais, em vez de uma convolução densa que misturasse todas as escalas. Para o mapa f e o

Quadro 2: Escalas temporais do banco de filtros e faixa de frequência que cada uma consegue representar a 128 Hz.

Escala	Comprimento	Duração	Representa a partir de	Faixa
1	5	39 ms	25,6 Hz	β alto
2	11	86 ms	11,6 Hz	β
3	15	117 ms	8,5 Hz	μ / α
4	33	258 ms	3,9 Hz	θ
5	65	508 ms	2,0 Hz	δ e potenciais lentos

Fonte: o autor (2026).

filtro espacial d , a operação é a combinação linear dos eletrodos dada pela Equação 3:

$$z_{f,d}[n] = \sum_{c=1}^3 v_{f,d}[c] \hat{h}_f[c, n], \quad (3)$$

em que $\hat{h}_f$ é o mapa temporal já normalizado. O agrupamento permite que cada faixa de frequência aprenda sua própria topografia, o que é razoável porque não há motivo para o ERD em μ e o ERD em β apresentarem a mesma distribuição entre C3 e C4. Trata-se do análogo aprendido de um filtro espacial do tipo CSP [6, 1]. A profundidade 3 é o teto útil da montagem: com três eletrodos não existem mais de três direções espaciais independentes. O resultado são 300 mapas de uma linha por 526 colunas.

1.5.3 Camada de potência

O bloco seguinte converte amplitude em potência de banda, e é a principal decisão de projeto da arquitetura. Após a convolução espacial, o sinal é elevado ao quadrado, submetido a uma média móvel de quatro amostras e passado por um logaritmo, conforme a Equação 4:

$$p_{f,d}[m] = \log \left(\max \left(\frac{1}{4} \sum_{n=4m}^{4m+3} z_{f,d}[n]^2, \varepsilon \right) \right), \quad \varepsilon = 10^{-6}. \quad (4)$$

Essa composição é a log-variância empregada no FBCSP [1], isto é, a potência de banda em janelas curtas, a grandeza física em que o ERD e o ERS são definidos [4]. O mesmo bloco aparece na ShallowFBCSPNet de [8], projetada com essa motivação. O piso ε evita que potências próximas de zero produzam $-\infty$ e anulem o gradiente.

Não há função de ativação retificadora antes do quadrado, e isso é deliberado:

o quadrado é a não-linearidade da cadeia. Retificar antes destruiria a medida de potência, porque a ReLU descarta o semiciclo negativo de uma oscilação e a média de um sinal oscilatório tende a zero. A versão anterior do classificador usava retificação seguida de *max pooling* nesse ponto, combinação inadequada para sinal oscilatório e substituída pelo bloco de potência pelo motivo apontado por [8].

1.5.4 Bloco separável e classificação

O último bloco convolucional é uma convolução separável, no padrão do segundo bloco da EEGNet [3]: primeiro uma convolução temporal de comprimento 15 aplicada a cada mapa isoladamente, que refina o eixo do tempo sem misturar mapas, e depois uma convolução pontual 1×1 que reduz os 300 mapas a 32. Seguem normalização em lote, retificação e um *pooling* adaptativo que fixa a saída em 32 posições temporais, tornando a rede independente do comprimento exato da janela. O vetor achatado de 1.024 elementos passa por *dropout* de 0,3 e por uma camada linear que produz os três *logits*. A conversão em probabilidades é feita pela função *softmax*, Equação 5:

$$P(y = i \mid x) = \frac{e^{o_i}}{\sum_{j=1}^3 e^{o_j}}, \quad i \in \{\text{rest, left, right}\}, \quad (5)$$

em que o_i é o *logit* da classe i . A classe predita é a de maior probabilidade, e o vetor completo de probabilidades é o que alimenta a regra de decisão descrita na Seção 1.9.

1.6 O que as convoluções extraem do sinal

Os filtros não representam os conceitos “mão esquerda” ou “mão direita”. O que se ajusta durante o treinamento são pesos que maximizam a separabilidade das três classes segundo a função de perda; o resultado é um conjunto de projeções temporais e espaciais cujas combinações discriminam as classes no conjunto de treino. A camada temporal aprende respostas em frequência: o painel (b) da Figura 3 mostra que os ramos de kernel longo concentram ganho na porção baixa do espectro e os de kernel curto se espalham até o teto da banda, comportamento coerente com o limite $128/k$. A camada espacial, por sua vez, aprende pesos relativos entre C3, C4 e Cz por faixa. A camada de potência converte essas projeções na grandeza em que o ERD é definido.

Isso não equivale a afirmar que um filtro específico corresponde a um fenômeno fisiológico determinado. A resposta em frequência exibida é uma visualização dos pesos treinados, não uma identificação de gerador cortical. A leitura fisiológica que o projeto sustenta é a de nível de sistema: a arquitetura foi construída para medir potência de banda por eletrodo e por faixa, que é onde a literatura localiza a assinatura da imagética motora [4, 5].

1.7 Treinamento

O modelo é treinado do zero, sem partir de pesos anteriores. Os checkpoints da versão anterior do projeto pertencem a outra arquitetura, de kernel único e retificação com *max pooling*, e a uma cadeia de pré-processamento que incluía a CAR; reaproveitá-los transferiria pesos ajustados a uma representação diferente. O Quadro 3 reúne os hiperparâmetros.

A perda ponderada, Equação 6, compensa o desbalanceamento das classes descrito na Seção 1.2:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \alpha_{y_i} \log P(y_i | x_i), \quad \alpha_c \propto \frac{1}{n_c}, \quad (6)$$

em que n_c é o número de amostras da classe c e os pesos α_c são normalizados para somar o número de classes.

O aumento de dados aplicado durante o treinamento é o conjunto de três transformações listado no Quadro 3, sorteadas de forma independente a cada apresentação da amostra e restritas ao conjunto de treino.

O aumento de dados sorteia suas transformações com o módulo `random` do Python, e não com o gerador do PyTorch. Semear apenas o PyTorch, portanto, deixava duas execuções nominalmente idênticas divergirem, e por isso os três geradores são semeados no início do treino. A repetição das gravações próprias por um fator 8 responde a uma segunda dificuldade: elas são 87 das 18.353 janelas, menos de 0,5% do conjunto, e desapareceriam no lote sem esse reforço. A repetição incide apenas sobre as 70 que caíram no treino, e nenhuma janela repetida entra na validação. A ressalva correspondente está na Seção 1.10.

A divisão em treino e validação é estratificada simultaneamente por classe e por

Quadro 3: Hiperparâmetros de treinamento.

Parâmetro	Valor
Inicialização	aleatória (sem <i>checkpoint</i> anterior)
Épocas	35
Tamanho do lote	16 no treino; 64 na validação, apenas para limitar o pico de memória
Otimizador	Adam, taxa de aprendizado 10^{-3}
Função de perda	entropia cruzada com pesos por classe, proporcionais ao inverso da frequência
Divisão treino/validação	80 %/20 %, estratificada por classe e por origem do dado: 14.683 e 3.670 janelas
Sementes	40 nos geradores do PyTorch, do Python e do NumPy
Aumento de dados	escala de amplitude entre 0,9 e 1,1; deslocamento temporal de até ± 16 amostras; ruído gaussiano de desvio 0,10, cada um com probabilidade 0,5, apenas no conjunto de treino
Repetição das gravações próprias	fator 8 no conjunto de treino: 490 janelas acrescentadas, levando o treino a 15.173
Seleção do modelo	melhor acurácia de validação ao longo das épocas (época 15, no treino do modelo distribuído)
Processamento	CPU

Fonte: o autor (2026), a partir de `config.py`.

origem, o que permite reportar a acurácia separadamente para o conjunto público e para as gravações próprias. As estatísticas de normalização são calculadas apenas sobre as amostras de treino, depois de definida a divisão, e gravadas no arquivo do modelo junto com os pesos e com a descrição da arquitetura. Esse último campo permite que o arquivo reconstrua a rede exatamente como foi treinada, mesmo que os valores padrão da classe mudem posteriormente.

1.8 Inferência sobre arquivos gravados

O comando `inferir_offline.py` carrega o modelo, reaplica a cadeia de pré-processamento e classifica os arquivos indicados. Aceita tanto os EDF do conjunto público, caso em que reconstrói as épocas a partir das anotações e compara com o rótulo verdadeiro, quanto os CSV capturados, caso em que recupera o rótulo do arquivo de eventos. A saída traz, por janela, a probabilidade de cada classe e a decisão, além da acurácia agregada e da matriz de confusão quando há rótulo disponível. Nas gravações contínuas o programa faz ainda uma segunda leitura, por janela deslizando, que varre o sinal inteiro sem usar os eventos e mede quantos comandos a órtese receberia. É o modo usado para avaliar o modelo e para produzir os números da Seção 1.10.

1.9 Inferência em tempo real com o capacete

A operação ao vivo é implementada em `tempo_real.py` e executa um laço com passo de 0,25 s. A cada iteração o programa solicita à placa as últimas 1.028 amostras, que a 250 Hz correspondem a 4,11 s, a janela que o modelo enxerga após a reamostragem para 128 Hz. Essa quantidade é calculada a partir do comprimento de janela lido do próprio arquivo do modelo, e não de um número fixo; com isso o *buffer* acompanha automaticamente qualquer mudança de janela ou de taxa.

O bloco lido passa pela mesma função usada para as gravações, que aplica o espelho nas bordas, a filtragem, a reamostragem e a remoção de tendência linear por canal. A normalização usa as estatísticas gravadas no modelo. A rede produz o vetor de probabilidades e a decisão é tomada pela regra da Equação 7:

$$\hat{y} = \begin{cases} \text{left,} & \text{se } P(\text{left} \mid x) > 0,50 \\ \text{right,} & \text{se } P(\text{right} \mid x) > 0,50 \text{ e } P(\text{left} \mid x) \leq 0,50 \\ \text{rest,} & \text{caso contrário.} \end{cases} \quad (7)$$

Entre o rótulo da janela isolada e o comando enviado à órtese está a classe `Decisor`, que trava o estado por 3,0 s depois de mandar fechar e impõe 2,0 s de tempo morto antes de um novo acionamento. A trava é necessária porque a evidência que a rede usa é o transiente da transição, que sai da janela deslizante poucos segundos depois do início do movimento; exigir confirmação contínua faria a mão reabrir sozinha enquanto o usuário ainda tenta fechá-la.

O limiar existe para conter disparos falsos, que numa órtese significam a mão fechando sozinha. A varredura por janela deslizante em seis *runs* contínuos do `PhysioNet` (44 tentativas, 740 s, passo de 0,25 s) produziu a relação registrada no `config.py`: com limiar 0,60, 73 % de detecção e um disparo falso a cada 16 s; com 0,70, 68 % e um a cada 29 s; com 0,80, 43 % e um a cada 53 s. Há uma divergência interna no pacote: essa tabela e os comentários do código apontam 0,80 como escolha, mas o valor atribuído a `config.LIMIAR` é 0,50, que é o que os programas usam enquanto a linha não for alterada.

O percurso de treino e o de operação diferem em poucos pontos: a origem do sinal, arquivo em um caso e transmissão contínua da placa no outro; o comprimento solicitado, ajustado ao modelo na operação ao vivo; e a decisão, que no treino é o simples máximo das probabilidades e na operação passa pelo limiar e pela trava. A filtragem, a reamostragem, o recorte, a remoção de tendência e a normalização são executados pelo mesmo código nos dois casos.

Para verificação sem o hardware de atuação, `tempo_real.py` com a opção `--sem-ortese` executa o mesmo laço sem abrir a porta serial, imprimindo apenas as probabilidades e a decisão; com `--simular-ortese`, imprime também os comandos que seriam enviados.

1.10 Integração com a órtese

Quando a decisão coincide com a classe configurada como alvo, o programa envia bytes pela porta serial a 115200 baud. O *firmware* do ESP32 mantém um laço de leitura da serial e interpreta quatro comandos, de um byte cada e sem terminador: R incrementa o ângulo até o teto de 180°, L decrementa até -30°, C leva o servo a 90°, que é o centro do curso e a posição de repouso da órtese, e um dígito de 0 a 9 define o tamanho do passo em graus. A cada byte recebido o firmware imprime `angle=<n>`, e o lado do PC lê esse eco e o usa como posição real, em vez de contar pulsos às cegas. O servomotor é acionado no GPIO 18, configurado a 50 Hz com largura de pulso entre 500 e 2500 μ s. O deslocamento entre o ângulo de repouso, 90°, e o de mão fechada, 120°, é executado em passos de 3° a cada 50 ms, portanto dez pulsos, e não de uma vez: o movimento gradual evita o golpe abrupto sobre a estrutura impressa e reduz o pico de corrente do atuador. Como o firmware não limita o curso à amplitude da órtese, essa limitação fica no PC, que recorta qualquer alvo ao intervalo entre repouso e fechado e devolve a órtese ao repouso ao encerrar.

Um segundo modo de acionamento, disponível em `inferir_offline.py` com a opção `--ortese`, classifica uma gravação já existente e aciona o microcontrolador durante a varredura, aplicando o mesmo limiar de probabilidade. Esse modo serve à demonstração controlada do encadeamento completo, com rótulo conhecido, e a opção `--simular-ortese` imprime os comandos que seriam enviados sem abrir a porta serial.

Referências

- [1] ANG, K. K. *et al.* Filter Bank Common Spatial Pattern (FBCSP) in brain-computer interface. **IEEE International Joint Conference on Neural Networks**, p. 2390–2397, 2008.
- [2] GOLDBERGER, A. L. *et al.* PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals. **Circulation**, v. 101, n. 23, p. e215–e220, 2000.

- [3] LAWHERN, V. J. *et al.* EEGNet: a compact convolutional neural network for EEG-based brain-computer interfaces. **Journal of Neural Engineering**, v. 15, n. 5, 056013, 2018.
- [4] PFURTSCHELLER, G.; LOPES DA SILVA, F. H. Event-related EEG/MEG synchronization and desynchronization: basic principles. **Clinical Neurophysiology**, v. 110, n. 11, p. 1842–1857, 1999.
- [5] PFURTSCHELLER, G.; NEUPER, C. Motor imagery and direct brain-computer communication. **Proceedings of the IEEE**, v. 89, n. 7, p. 1123–1134, 2001.
- [6] RAMOSER, H.; MÜLLER-GERKING, J.; PFURTSCHELLER, G. Optimal spatial filtering of single trial EEG during imagined hand movement. **IEEE Transactions on Rehabilitation Engineering**, v. 8, n. 4, p. 441–446, 2000.
- [7] SCHALK, G. *et al.* BCI2000: a general-purpose brain-computer interface (BCI) system. **IEEE Transactions on Biomedical Engineering**, v. 51, n. 6, p. 1034–1043, 2004.
- [8] SCHIRRMESTER, R. T. *et al.* Deep learning with convolutional neural networks for EEG decoding and visualization. **Human Brain Mapping**, v. 38, n. 11, p. 5391–5420, 2017.